



École Nationale des

Sciences Appliquées d'Al Hoceima

Filière : TDIA – 2^{ème} Année

Rapport TP – JEE MVC1

Gestion des Produits avec Au-
thentification et Gestion des Rôles

Réalisé par :

Yassir HAMRI

Encadré par :

Pr. Mohamed CHERRADI

Année Universitaire : 2025/2026

Table des matières

1	Création du projet et configuration initiale	3
1.1	Arborescence du Projet	3
2	DAO (sans user)	3
2.1	Produit.java – L’entité principale	3
2.2	produitDAO.java – L’interface du contrat	4
2.3	produitImpl.java – Implémentation en mémoire	4
3	Service	5
3.1	produitMetier.java – L’interface métier	5
3.2	produitMetierImpl.java – Le Singleton	6
4	Servlets CRUD	6
4.1	AddProduitServlet.java	6
4.2	DeleteProduitServlet.java	7
4.3	EditProduitServlet.java	7
4.4	UpdateProduitServlet.java	8
4.5	ListProduitServlet.java (version initiale)	8
5	Première JSP (index.jsp)	9
6	User DAO	10
6.1	user.java – Le modèle utilisateur	10
6.2	userDAO.java – La gestion des comptes	10
7	Authentification	11
7.1	loginServlet.java	11
8	Modification du ListProduitServlet	12
9	logoutServlet	12
10	Filtre d’authentification	13
10.1	AuthFilter.java	13
11	login.jsp	14
12	list.jsp	14
13	Modification index.jsp	15
14	Gestion des erreurs	15
14.1	error.jsp – La Page d’Erreur Centralisée	15
14.2	Configuration erreurs dans web.xml	15
15	Configuration finale web.xml	16
16	Conclusion	16

1 Création du projet et configuration initiale

Le projet a été structuré en plusieurs packages :

- dao
- services
- web
- filter

Le choix a été fait d'utiliser les annotations `@WebServlet` et `@WebFilter` plutôt qu'une déclaration exhaustive dans `web.xml`, afin de garder la configuration au plus proche du code concerné.

Ce projet consiste à développer une application web dynamique de gestion de produits, reposant sur l'architecture **MVC1** (une servlet par action) et le modèle **3-Tiers**. Pour le TP4, le système a été complété par un mécanisme d'authentification, une gestion de rôles (Admin / Utilisateur), un filtre de sécurité, et une gestion centralisée des erreurs.

1.1 Arborescence du Projet

```
src/main
|-- java
|   |-- dao
|   |   |-- Produit.java           (Entite produit)
|   |   |-- produitDAO.java       (Interface DAO produit)
|   |   |-- produitImpl.java      (Implementation DAO + donnees initiales)
|   |   |-- user.java            (Entite utilisateur avec role)
|   |   |-- userDAO.java          (Verification des identifiants)
|   |-- services
|   |   |-- produitMetier.java     (Interface service)
|   |   |-- produitMetierImpl.java (Singleton - couche service)
|   |-- filter
|   |   |-- AuthFilter.java        (Filtre de securite)
|   |-- web
|   |   |-- AddProduitServlet.java
|   |   |-- DeleteProduitServlet.java
|   |   |-- EditProduitServlet.java
|   |   |-- UpdateProduitServlet.java
|   |   |-- ListProduitServlet.java
|   |   |-- loginServlet.java
|   |   |-- logoutServlet.java
|   |-- webapp
|   |   |-- index.jsp             (Vue Admin - CRUD complet)
|   |   |-- list.jsp             (Vue User - lecture seule)
|   |   |-- login.jsp            (Formulaire de connexion)
|   |   |-- error.jsp            (Page d'erreur)
|   |-- WEB-INF
|   |-- web.xml
```

2 DAO (sans user)

2.1 Produit.java – L'entité principale

La classe `Produit` est le modèle central. Elle suit la convention **Java Bean** : attributs privés et accès via des getters/setters publics. Cette convention est indispensable pour l'Expression Language (EL) des JSP, car `${p.nom}` appelle automatiquement `p.getNom()`.

Listing 1 – dao/Produit.java

```
1 package dao;
2 public class Produit {
3     private Long idProduit;
4     private String nom;
5     private String description;
6     private Double prix;
7     public Produit() {}
8     public Produit(String nom, String s, Double p) {
9         this.nom = nom;
10        this.description = s;
11        this.prix = p;
12    }
13    public Long getIdProduit() { return idProduit; }
14    public void setIdProduit(Long idProduit) { this.idProduit = idProduit; }
15    public String getNom() { return nom; }
16    public void setNom(String nom) { this.nom = nom; }
17    public String getDescription() { return description; }
18    public void setDescription(String description) { this.description =
19        description; }
20    public Double getPrix() { return prix; }
21    public void setPrix(Double prix) { this.prix = prix; }
22 }
```

2.2 produitDAO.java – L'interface du contrat

L'interface produitDAO définit le contrat que toute implémentation doit respecter. Ce principe garantit un **couplage faible** : la couche service ne connaît que l'interface, pas l'implémentation concrète.

Listing 2 – dao/produitDAO.java

```
1 package dao;
2 import java.util.*;
3 public interface produitDAO {
4     public void addProduit(Produit p);
5     public void deleteProduit(Long id);
6     public void editProduit(Produit p);
7     public void updateProduit(Produit p);
8     public List<Produit> getAllProduits();
9     public Produit getProduitById(Long ID);
10 }
```

2.3 produitImpl.java – Implémentation en mémoire

produitImpl est l'implémentation concrète de l'interface DAO. Elle utilise une ArrayList en mémoire. La méthode init() est appelée une seule fois au démarrage pour pré-charger deux produits par défaut.

Listing 3 – dao/produitImpl.java

```
1 package dao;
2 import java.util.*;
3 public class produitImpl implements produitDAO {
```

```

4 private List<Produit> products = new ArrayList<>();
5 public void init() {
6 addProduit(new Produit("PC 1", "Sony vaio 1", 7000.0));
7 addProduit(new Produit("PC 2", "Sony vaio 2", 6000.0));
8 }
9 public void addProduit(Produit p) {
10 p.setIdProduit(new Long(products.size() + 1));
11 products.add(p);
12 }
13 public void deleteProduit(Long id) {
14 products.remove(getProduitById(id));
15 }
16 public Produit getProduitById(Long id) {
17 for (Produit p : products) {
18 if (p.getIdProduit().equals(id)) return p;
19 }
20 return null;
21 }
22 public List<Produit> getAllProduits() { return products; }
23 public void updateProduit(Produit p) {
24 for (int i = 0; i < products.size(); i++) {
25 Produit ep = products.get(i);
26 if (ep.getIdProduit().equals(p.getIdProduit())) {
27 ep.setNom(p.getNom());
28 ep.setDescription(p.getDescription());
29 ep.setPrix(p.getPrix());
30 break;
31 }
32 }
33 }
34 @Override
35 public void editProduit(Produit p) {}
36 }

```

3 Service

3.1 produitMetier.java – L'interface métier

La couche service joue le rôle d'intermédiaire entre les servlets et le DAO. L'interface produitMetier expose exactement les mêmes opérations que le DAO, renforçant l'indépendance des couches.

Listing 4 – services/produitMetier.java

```

1 package services;
2 import dao.*;
3 import java.util.*;
4 public interface produitMetier {
5 public void addProduit(Produit p);
6 public void deleteProduit(Long id);
7 public void editProduit(Produit p);
8 public void updateProduit(Produit p);
9 public List<Produit> getAllProduits();
10 public Produit getProduitById(Long ID);

```

11 }

3.2 produitMetierImpl.java – Le Singleton

L'implémentation est réalisée selon le **patron de conception Singleton** : une seule instance de la classe est créée et partagée entre toutes les servlets. Ceci est crucial pour que toutes les servlets opèrent sur la *même* liste de produits en mémoire. La méthode `getInstance()` crée l'instance seulement si elle n'existe pas encore.

Listing 5 – services/produitMetierImpl.java

```
1 package services;
2 import dao.*;
3 import java.util.*;
4 public class produitMetierImpl implements produitMetier {
5     public static produitMetierImpl instance;
6     private produitDAO dao;
7     private produitMetierImpl() {
8         dao = new produitImpl();
9         ((produitImpl) dao).init(); // Charge les produits par défaut
10    }
11    public static produitMetierImpl getInstance() {
12        if (instance == null) {
13            instance = new produitMetierImpl();
14        }
15        return instance;
16    }
17    @Override public void addProduit(Produit p) { dao.addProduit(p); }
18    @Override public void deleteProduit(Long id) { dao.deleteProduit(id); }
19    @Override public void editProduit(Produit p) { dao.editProduit(p); }
20    @Override public void updateProduit(Produit p) { dao.updateProduit(p); }
21    @Override public List<Produit> getAllProduits() { return dao.getAllProduits(); }
22    @Override public Produit getProduitById(Long ID) { return dao.getProduitById(ID); }
23 }
```

4 Servlets CRUD

Chaque servlet est responsable d'une unique opération, conformément au principe MVC1. Toutes utilisent l'annotation `@WebServlet` pour se déclarer sans passer par le `web.xml`.

4.1 AddProduitServlet.java

Traite la soumission du formulaire d'ajout via `doPost`. Les paramètres du formulaire sont récupérés avec `request.getParameter()`, puis un objet `Produit` est construit et transmis au service.

Listing 6 – web/AddProduitServlet.java

```
1 @WebServlet("/addProduit")
2 public class AddProduitServlet extends HttpServlet {
3     private static final produitMetier metier = produitMetierImpl.
        getInstance();
4     protected void doPost(HttpServletRequest request,
5         HttpServletResponse response)
6         throws ServletException, IOException {
7         String nom = request.getParameter("nom");
8         String description = request.getParameter("description");
9         Double prix = Double.parseDouble(request.getParameter("prix"));
10        Produit p = new Produit(nom, description, prix);
11        metier.addProduit(p);
12        request.setAttribute("listeProduits", metier.getAllProduits());
13        request.getRequestDispatcher("index.jsp").forward(request, response);
14    }
15 }
```

4.2 DeleteProduitServlet.java

Reçoit l'identifiant du produit via un paramètre GET dans l'URL (?id=X), supprime le produit, puis redirige vers listProduits pour recharger la liste à jour.

Listing 7 – web/DeleteProduitServlet.java

```
1 @WebServlet("/deleteProduit")
2 public class DeleteProduitServlet extends HttpServlet {
3     private static final produitMetier metier = produitMetierImpl.
        getInstance();
4     protected void doGet(HttpServletRequest request,
5         HttpServletResponse response)
6         throws ServletException, IOException {
7         Long id = Long.parseLong(request.getParameter("id"));
8         metier.deleteProduit(id);
9         response.sendRedirect("listeProduits");
10    }
11 }
```

4.3 EditProduitServlet.java

Prépare l'écran de modification : elle charge le produit à éditer et le place dans la requête sous l'attribut produitEdit. La JSP index.jsp détecte ensuite cet attribut pour pré-remplir les champs du formulaire.

Listing 8 – web/EditProduitServlet.java

```
1 @WebServlet("/editProduit")
2 public class EditProduitServlet extends HttpServlet {
3     private static final produitMetier metier = produitMetierImpl.
        getInstance();
4     protected void doGet(HttpServletRequest request,
5         HttpServletResponse response)
6         throws ServletException, IOException {
7         Long id = Long.parseLong(request.getParameter("id"));
```

```

8  Produit p = metier.getProduitById(id);
9  request.setAttribute("produitEdit", p);
10 request.setAttribute("listeProduits", metier.getAllProduits());
11 request.getRequestDispatcher("index.jsp").forward(request, response);
12 }
13 }

```

4.4 UpdateProduitServlet.java

Reçoit les nouvelles valeurs via le formulaire POST, reconstruit un objet Produit avec les données modifiées, et demande au service de procéder à la mise à jour dans la liste.

Listing 9 – web/UpdateProduitServlet.java

```

1  @WebServlet("/updateProduit")
2  public class UpdateProduitServlet extends HttpServlet {
3  private static produitMetierImpl metier = produitMetierImpl.getInstance
4      ();
5  protected void doPost(HttpServletRequest request,
6      HttpServletResponse response)
7      throws ServletException, IOException {
8  Long id = Long.parseLong(request.getParameter("idProduit"));
9  String nom = request.getParameter("nom");
10 String desc = request.getParameter("description");
11 Double prix = Double.parseDouble(request.getParameter("prix"));
12 Produit p = new Produit();
13 p.setIdProduit(id);
14 p.setNom(nom);
15 p.setDescription(desc);
16 p.setPrix(prix);
17 metier.updateProduit(p);
18 request.setAttribute("listeProduits", metier.getAllProduits());
19 request.getRequestDispatcher("index.jsp").forward(request, response);
20 }

```

4.5 ListProduitServlet.java (version initiale)

Dans la version initiale, sans gestion de rôle, la servlet charge simplement la liste des produits et transfère systématiquement vers index.jsp.

Listing 10 – web/ListProduitServlet.java – version initiale

```

1  protected void doGet(HttpServletRequest request,
2      HttpServletResponse response)
3      throws ServletException, IOException {
4  String idProduit = request.getParameter("idProduit");
5  List<Produit> liste = new ArrayList<>();
6  if (idProduit != null && !idProduit.isEmpty()) {
7  try {
8  Long id = Long.parseLong(idProduit);
9  Produit p = metier.getProduitById(id);
10 if (p != null) liste.add(p);
11 } catch (NumberFormatException e) {
12 liste = metier.getAllProduits();

```



```

13 }
14 } else {
15     liste = metier.getAllProduits();
16 }
17 request.setAttribute("listeProduits", liste);
18 // Version initiale : transfert unique vers index.jsp
19 request.getRequestDispatcher("index.jsp").forward(request, response);
20 }

```

5 Première JSP (index.jsp)

Dans sa version initiale, `index.jsp` est une vue CRUD simple, sans affichage du compte connecté ni lien de déconnexion. La directive `errorPage` et la ligne de bienvenue seront ajoutées plus tard.

Listing 11 – `webapp/index.jsp` – version initiale

```

1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2  <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
3  <html>
4  <head><title>Gestion des Produits MVC1</title></head>
5  <body>
6  <h2>Gestion des Produits (MVC1)</h2>
7
8  <form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}"
9  method="post">
10 <input type="hidden" name="idProduit"
11 value="${produitEdit.idProduit}" />
12 Nom: <input type="text" name="nom"
13 value="${produitEdit.nom}" required />
14 Description: <input type="text" name="description"
15 value="${produitEdit.description}" required />
16 Prix: <input type="text" name="prix"
17 value="${produitEdit.prix}" required />
18 <input type="submit"
19 value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />
20 </form>
21 <hr/>
22 <form action="listProduits" method="get">
23 ID: <input type="text" name="idProduit" />
24 <input type="submit" value="Rechercher" />
25 </form>
26 <hr/>
27 <table border="1" cellpadding="5">
28 <tr>
29 <th>ID</th><th>Nom</th><th>Description</th><th>Prix</th><th>Actions</th>
30 </tr>
31 <c:forEach var="p" items="${listeProduits}">
32 <tr>
33 <td>${p.idProduit}</td>
34 <td>${p.nom}</td>
35 <td>${p.description}</td>
36 <td>${p.prix}</td>
37 <td>
38 <a href="editProduit?id=${p.idProduit}">Modifier</a> |

```

```

39 <a href="deleteProduit?id=${p.idProduit}"
40 onclick="return confirm('Supprimer ?');">
41 Supprimer
42 </a>
43 </td>
44 </tr>
45 </c:forEach>
46 </table>
47 </body>
48 </html>

```

6 User DAO

6.1 user.java – Le modèle utilisateur

La classe user représente un compte applicatif. Elle contient trois champs : login, password et role. Le role déterminera quelle interface sera présentée après la connexion.

Listing 12 – dao/user.java

```

1 package dao;
2 public class user {
3     private String login;
4     private String password;
5     private String role;
6     public user() {}
7     public user(String login, String password, String role) {
8         this.login = login;
9         this.password = password;
10        this.role = role;
11    }
12    public String getLogin() { return login; }
13    public void setLogin(String login) { this.login = login; }
14    public String getPassword() { return password; }
15    public void setPassword(String password) { this.password = password; }
16    public String getRole() { return role; }
17    public void setRole(String role) { this.role = role; }
18 }

```

6.2 userDAO.java – La gestion des comptes

userDAO dispose d'une liste static d'utilisateurs initialisée dans un bloc static{}. Ce bloc s'exécute une seule fois au chargement de la classe, permettant de pré-enregistrer un administrateur et un utilisateur standard.

Listing 13 – dao/userDAO.java

```

1 package dao;
2 import java.util.*;
3 public class userDAO {
4     private static List<user> users = new ArrayList<>();
5     static {
6         users.add(new user("admin", "1234", "admin"));

```

```

7  users.add(new user("user", "1234", "user"));
8  }
9  public user findByLoginAndPassword(String login, String password) {
10 for (user u : users) {
11 if (u.getLogin().equals(login) && u.getPassword().equals(password)) {
12 return u;
13 }
14 }
15 return null;
16 }
17 }

```

7 Authentification

7.1 loginServlet.java

La servlet de connexion expose deux comportements : en GET, elle affiche le formulaire via un forward vers login.jsp. En POST, elle vérifie les identifiants via le userDAO. Si la vérification réussit, l'objet user est stocké en session sous la clé "user", puis l'utilisateur est redirigé vers listProduits (qui se chargera de l'orienter selon son rôle).

Listing 14 – web/loginServlet.java

```

1  @WebServlet("/login")
2  public class loginServlet extends HttpServlet {
3  private userDAO dao = new userDAO();
4  protected void doGet(HttpServletRequest request,
5  HttpServletResponse response)
6  throws ServletException, IOException {
7  request.getRequestDispatcher("login.jsp").forward(request, response);
8  }
9  protected void doPost(HttpServletRequest request,
10 HttpServletResponse response)
11 throws ServletException, IOException {
12 String login = request.getParameter("login");
13 String password = request.getParameter("password");
14 user u = dao.findByLoginAndPassword(login, password);
15 if (u != null) {
16 HttpSession session = request.getSession();
17 session.setAttribute("user", u);
18 if ("admin".equals(u.getRole())) {
19 response.sendRedirect("listProduits");
20 } else {
21 response.sendRedirect("listProduits");
22 }
23 } else {
24 request.setAttribute("error", "login or pwd is incorrect");
25 request.getRequestDispatcher("login.jsp").forward(request, response);
26 }
27 }
28 }

```

Points clés :

- `request.getSession()` crée une session HTTP et lui associe le cookie JSESSIONID côté navigateur.
- La structure `if/else` sur le rôle est maintenue pour montrer explicitement la gestion différenciée, même si les deux branches redirigent vers `listProduits` qui dispatche lui-même.
- En cas d'échec, `forward` est utilisé (et non `redirect`) pour que l'attribut "error" reste accessible dans la JSP.

8 Modification du ListProduitServlet

Pour intégrer la gestion des rôles, les trois lignes suivantes ont été ajoutées à la fin du `doGet`, remplaçant l'ancien `forward` fixe vers `index.jsp` :

- Récupération de la session sans en créer une nouvelle (`getSession(false)`)
- Cast de l'attribut session "user" en objet `user`
- Dispatch conditionnel : `admin` → `index.jsp`, sinon → `list.jsp`

Listing 15 – `web/ListProduitServlet.java` – version modifiée avec rôles

```

1 request.setAttribute("listeProduits", liste);
2
3 // Ajout TP4 : dispatch selon le role
4 HttpSession session = request.getSession(false);
5 user u = (user) session.getAttribute("user");
6
7 if (u != null && "admin".equals(u.getRole())) {
8     request.getRequestDispatcher("index.jsp").forward(request, response)
9     ;
10 } else {
11     request.getRequestDispatcher("list.jsp").forward(request, response);
12 }
```

La méthode `getSession(false)` est intentionnelle : elle ne crée pas une nouvelle session si elle n'existe pas déjà. Sans cela, un utilisateur non connecté verrait une session vide créée inutilement. Le cast `(user) session.getAttribute("user")` est obligatoire car `getAttribute` retourne un `Object` générique.

9 logoutServlet

La déconnexion consiste à invalider la session active. `getSession(false)` est utilisé explicitement pour ne *pas* créer une nouvelle session si aucune n'existe – ce qui serait logiquement contradictoire lors d'une déconnexion.

Listing 16 – `web/logoutServlet.java`

```

1 @WebServlet("/logout")
2 public class logoutServlet extends HttpServlet {
3     protected void doGet(HttpServletRequest request,
4         HttpServletResponse response)
5         throws ServletException, IOException {
6         HttpSession session = request.getSession(false);
```

```

7  if (session != null) {
8  session.invalidate();
9  }
10 response.sendRedirect("login");
11 }
12 }

```

10 Filtre d'authentification

10.1 AuthFilter.java

Le filtre est le gardien de toutes les URLs protégées. Conformément au cours (Chapitre 2, page 272), il intercepte chaque requête vers les servlets de gestion de produits et vérifie la présence d'un attribut "user" en session. Sans lui, un utilisateur pourrait accéder directement aux URLs sans passer par la page de connexion. Le filtre est déclaré dans web.xml (voir section suivante) plutôt qu'avec une annotation, pour centraliser la configuration de sécurité dans le descripteur de déploiement.

Listing 17 – filter/AuthFilter.java

```

1  package filter;
2  public class AuthFilter extends HttpFilter implements Filter {
3  public void doFilter(ServletRequest req, ServletResponse resp,
4  FilterChain chain)
5  throws IOException, ServletException {
6  HttpServletRequest request = (HttpServletRequest) req;
7  HttpServletResponse response = (HttpServletResponse) resp;
8  HttpSession session = request.getSession(false);
9  if (session != null && session.getAttribute("user") != null) {
10 chain.doFilter(req, resp); // Session valide : on laisse passer
11 } else {
12 // Pas de session : redirection vers la page de login
13 response.sendRedirect(request.getContextPath() + "/login");
14 }
15 }
16 public void init(FilterConfig fConfig) throws ServletException {}
17 public void destroy() {}
18 }

```

- Les paramètres req et resp sont de type générique (ServletRequest et ServletResponse). Le **cast** vers leurs versions HTTP est obligatoire pour pouvoir appeler getSession() et sendRedirect(), méthodes spécifiques au protocole HTTP.
- getSession(false) ne crée jamais de nouvelle session. Si aucune session n'existe, il retourne null – ce que l'on teste immédiatement.
- chain.doFilter(req, resp) passe le relais au prochain filtre ou directement à la servlet cible. Ne pas appeler cette méthode bloque définitivement la requête.
- getContextPath() retourne le préfixe du contexte (ex : /GestionDesProduitsMVC1), indispensable pour construire une URL de redirection absolue correcte.

11 login.jsp

La page de login n'utilise pas de JSTL mais un scriptlet Java classique pour afficher le message d'erreur. L'attribut "error" est placé dans la requête par loginServlet en cas d'échec d'authentification.

Listing 18 – webapp/login.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <html>
3 <head><title>Login</title></head>
4 <body>
5 <h2>Login</h2>
6 <% String erreur = (String) request.getAttribute("error");
7 if (erreur != null) { %>
8 <p style="color:red;"><%= erreur %></p>
9 <% } %>
10 <form action="login" method="post">
11 Login: <input type="text" name="login" required /><br/>
12 Mot de passe: <input type="password" name="password" required /><br/>
13 <input type="submit" value="Se connecter" />
14 </form>
15 </body>
16 </html>
```

12 list.jsp

Cette vue est strictement réservée aux utilisateurs avec le rôle "user". Aucun formulaire d'ajout, aucun lien de modification ou de suppression n'y est présent. Elle affiche uniquement la liste en lecture seule.

Listing 19 – webapp/list.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java"
2 errorPage="error.jsp" %>
3 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4 <html>
5 <head><title>Liste des Produits</title></head>
6 <body>
7 <h2>Liste des Produits</h2>
8 <p>Bienvenue, ${user.login} | <a href="logout">Deconnexion</a></p>
9 <table border="1" cellpadding="5">
10 <tr>
11 <th>ID</th><th>Nom</th><th>Description</th><th>Prix</th>
12 </tr>
13 <c:forEach var="p" items="${listeProduits}">
14 <tr>
15 <td>${p.idProduit}</td>
16 <td>${p.nom}</td>
17 <td>${p.description}</td>
18 <td>${p.prix}</td>
19 </tr>
20 </c:forEach>
21 </table>
22 </body>
23 </html>
```

13 Modification index.jsp

Deux modifications ont été apportées à la version initiale d'`index.jsp` pour intégrer l'authentification :

1. Ajout de `errorPage="error.jsp"` dans la directive `page` : toute exception non capturée sera interceptée et redirigée vers la page d'erreur centralisée.
2. Ajout de la ligne de bienvenue avec le login et le lien de déconnexion, exploitant l'Expression Language pour accéder directement à l'objet `user` mis en session.

Listing 20 – Lignes ajoutées dans `index.jsp` pour le TP4

```
1 <!-- Ligne 1 : directive modifiée -->
2 <%@ page contentType="text/html; charset=UTF-8" language="java"
3     errorPage="error.jsp" %>
4
5 <!-- Ligne 2 : ajoutée après le titre -->
6 <p>Bienvenue, ${user.login} | <a href="logout">Déconnexion</a></p>
```

`${user.login}` exploite l'Expression Language (EL) : le conteneur JSP cherche automatiquement un attribut nommé `user` dans les scopes (request, session, application) et appelle `getLogin()` dessus. L'objet `user` a été placé en session par `loginServlet` via `session.setAttribute("user", u)`, ce qui le rend accessible ici sans import ni scriptlet Java.

14 Gestion des erreurs

14.1 error.jsp – La Page d'Erreur Centralisée

La directive `isErrorPage="true"` donne à cette JSP l'accès à la variable implicite `exception`, disponible uniquement dans les pages d'erreur. La directive `errorPage="error.jsp"` présente dans `index.jsp` et `list.jsp` désigne cette page comme destination en cas d'exception non capturée.

Listing 21 – `webapp/error.jsp`

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java"
2 isErrorPage="true" %>
3 <html>
4 <head><title>Erreur</title></head>
5 <body>
6 <h2>Une erreur est survenue</h2>
7 <p style="color:red;"><%= exception.getMessage() %></p>
8 <a href="listProduits">Retour à la liste</a>
9 </body>
10 </html>
```

14.2 Configuration erreurs dans `web.xml`

Capture des erreurs HTTP 404 et 500 pour afficher `error.jsp` plutôt que la page d'erreur par défaut de Tomcat.

15 Configuration finale web.xml

Le fichier web.xml contient trois éléments de configuration clés que les annotations ne peuvent pas gérer directement :

1. **Welcome file** : redirige l'accès à la racine du projet vers login.
2. **Error pages** : capture les erreurs HTTP 404 et 500 pour afficher error.jsp.
3. **Filtre** : enregistre AuthFilter et définit les URLs qu'il doit protéger.

Ajouts :

- welcome-file
- filter mapping

Listing 22 – WEB-INF/web.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app ...>
3 <display-name>GestionDesProduitsMVC1</display-name>
4 <welcome-file-list>
5 <welcome-file>login</welcome-file>
6 <welcome-file>login.jsp</welcome-file>
7 </welcome-file-list>
8 <error-page>
9 <error-code>404</error-code>
10 <location>/error.jsp</location>
11 </error-page>
12 <error-page>
13 <error-code>500</error-code>
14 <location>/error.jsp</location>
15 </error-page>
16 <filter>
17 <filter-name>AuthFilter</filter-name>
18 <filter-class>filter.AuthFilter</filter-class>
19 </filter>
20 <filter-mapping>
21 <filter-name>AuthFilter</filter-name>
22 <url-pattern>/listProduits</url-pattern>
23 <url-pattern>/addProduit</url-pattern>
24 <url-pattern>/deleteProduit</url-pattern>
25 <url-pattern>/editProduit</url-pattern>
26 <url-pattern>/updateProduit</url-pattern>
27 <url-pattern>/index.jsp</url-pattern>
28 <url-pattern>/list.jsp</url-pattern>
29 </filter-mapping>
30 </web-app>
```

16 Conclusion

Ce projet illustre de manière concrète le fonctionnement de l'architecture MVC1 dans un contexte Java EE. Chaque couche a un rôle bien défini : le DAO gère l'accès aux données, le service assure la logique métier via un Singleton, les servlets traitent les requêtes HTTP et les vues JSP/JSTL affichent les résultats sans code Java superflu. L'ajout de l'authentification et de la gestion des rôles pour le TP4 a permis de mettre en

pratique deux mécanismes fondamentaux supplémentaires : les sessions HTTP pour maintenir l'état de connexion entre les requêtes, et les filtres de servlets pour protéger les ressources de manière centralisée et transparente – exactement comme le recommande le Chapitre 2 du cours.